# Marking Scheme: Perform Programming Principles (0613 451 05A) – CAT 2

**Aberdare Technical College – Information Communication Technology** | Time: 1 hr 30 min | Total: 50 marks

**Notes to the marker:** Award marks for correct alternative answers and equivalent wording. For programs, accept any logically correct solution; do not penalise minor syntax slips (e.g. a missing semicolon) more than once per program. Award follow-through marks where a candidate's earlier error is carried forward consistently.

## Question 1 (3 marks)

- Definition: a control structure is a programming construct that determines/controls the order (flow) in which statements in a program are executed. **(1)**
- Three types: **Sequence, Selection (decision/branching), Iteration (looping/repetition)**. **(2)** All three correct = 2 marks; any one or two correct = 1 mark.

## Question 2 (3 marks)

Any three, 1 mark each (function name + what it reads):

- `scanf()` – reads formatted input (integers, floats, characters, words) of various types
- `getchar()` – reads a single character
- `gets()` / `fgets()` – reads a line/string of text
- `getch()` / `getche()` – reads a single character without waiting for Enter (accept)

## Question 3 (4 marks)

Any four, 1 mark each:

- Range of values to be stored (smallest to largest)
- Memory/storage size required (bytes)
- Precision/accuracy needed (whole vs. decimal numbers, number of digits)
- Nature/type of data (numeric, character, text)
- Signed or unsigned values (whether negatives are needed)

- Operations to be performed / processing speed and efficiency
- Portability/compatibility with the system or hardware

## Question 4 (3 marks)

- **Function declaration (prototype):** tells the compiler the function's name, return type and parameter types before it is used; has no body and ends with a semicolon, e.g. `int add(int, int);` **(1)**
- **Function definition:** the actual implementation of the function, containing the function header and the body (statements) that perform the task, enclosed in `{ }`. **(1)**
- One clear difference (e.g. the declaration has no body and allocates no code, the definition contains the body; the declaration is placed before `main()`/at the top, the definition provides the logic). **(1)**

## Question 5 (3 marks)

- (a) `marks[2]` = **70 (1)**; `marks[4]` = **90 (1)**
- (b) First valid index = **0**, last valid index = **4 (1)** (both needed)

## Question 6 (4 marks)

- (a) For `int a[2][3][4]`:
  - First index (size 2): selects the block/table/plane, valid values 0–1 **(1)**
  - Second index (size 3): selects the row within a block, valid values 0–2 **(1)**
  - Third index (size 4): selects the column within a row, valid values 0–3 **(1)**
- (b) Total elements = 2 × 3 × 4 = **24 (1)**

## Question 7 (5 marks)

- (a) Rows = **3 (1)**; columns = **4 (1)**
- (b) `scores[1][1]` = **18 (1)**; `scores[2][3]` = **5 (1)**
- (c) `scores[2][0] = 30;` **(1)** (Indexes start at 0, so row 2 is the third row. Accept `scores[2][0] = 30;` only; award the mark for correct syntax with a semicolon.)

## Question 8 (5 marks)

- (a) Program 1 output: **Number = 10 (1)**; Program 2 output: **Number = 50 (1)**
- (b) Program 1: **pass by value (1)**; Program 2: **pass by reference** (by address/using pointers) **(1)**
- (c) In Program 1 the function works on a copy of the variable, so the original is unchanged; in Program 2 the function receives the address of the variable and modifies the original value. **(1)**

## Question 9 (5 marks)

1 mark each: correct data type plus a valid reason (both needed for the mark).

- (a) Student's age – `int` (or `unsigned char` / `short`): whole number, small positive range
- (b) Population above 2.1 billion – `long long` / `unsigned long` / `unsigned int` (accept `long`): `int` max is about 2,147,483,647, so a larger range is needed
- (c) Percentage 0–100 on limited memory – `unsigned char` (or `char`): uses only 1 byte and covers 0–255
- (d) About 15 digits of precision – `double`: gives roughly 15–16 significant digits (`float` has only about 6–7)
- (e) Single grade letter – `char`: holds one character using 1 byte

## Question 10 (7 marks)

| Marking point | Marks |
|---|---|
| `#include <stdio.h>` and a correct `main()` structure | 1 |
| Declaration of a suitable variable for marks (`int` or `float`) | 1 |
| Prompt and input using `scanf()` with a correct format specifier and `&` | 1 |
| `if (marks >= 80)` displays Grade A | 1 |

| Marking point | Marks |
|---|---|
| `else if` for 60–79 displays Grade B (condition correct) | 1 |
| `else if` for 50–59 displays Grade C (condition correct) | 1 |
| Final `else` displays Fail; program syntactically correct (braces, `return 0`) | 1 |

**Model answer:**

```
#include <stdio.h>

int main()
{
    int marks;

    printf("Enter the student's marks: ");
    scanf("%d", &marks);

    if (marks >= 80)
        printf("Grade A");
    else if (marks >= 60)
        printf("Grade B");
    else if (marks >= 50)
        printf("Grade C");
    else
        printf("Fail");

    return 0;
}
```

Accept equivalent conditions such as `marks >= 60 && marks <= 79`.

# Question 11 (8 marks)

| Marking point | Marks |
|---|---|
| (a) Function header with `int` return type, name `multiply` and two `int` parameters | 1 |
| (a) Product computed correctly (`a * b`) | 1 |
| (a) Result returned using `return` | 1 |
| (b) Correct prototype before `main()`, e.g. `int multiply(int, int);` ending with semicolon | 1 |
| (c) Variables declared and prompt(s) displayed | 1 |
| (c) Two integers read using `scanf()` with correct format and `&` | 1 |
| (c) Function called with correct arguments and result stored/used | 1 |
| (c) Result displayed using `printf()` | 1 |

**Model answer:**

```
#include <stdio.h>

int multiply(int a, int b);   /* function prototype */

int main()
{
    int x, y, result;

    printf("Enter two integers: ");
    scanf("%d %d", &x, &y);

    result = multiply(x, y);   /* function call */
    printf("Product = %d", result);

    return 0;
}

/* function definition */
```

```
int multiply(int a, int b)
{
    return a * b;
}
```

# Mark Summary

| Question | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | **Total** |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Marks | 3 | 3 | 4 | 3 | 3 | 4 | 5 | 5 | 5 | 7 | 8 | **50** |